

PARAMETER-BLOCKED LANCZOS SENSITIVITY REPLAY FOR LOW-LAUNCH MATRIX-FUNCTION GRADIENTS

MOLENA HUYNH

North Carolina State University
molena.huynh@jmp.com

Abstract. Differentiating a finite Lanczos matrix-function objective by tangent replay usually processes one parameter direction at a time. That organization is memory economical but launches the Hermitian operator twice for every parameter and Krylov step. We introduce parameter-blocked tridiagonal sensitivity replay. A block of q tangent directions is propagated as an $n \times q$ multivector while the primal Lanczos recurrence is replayed once. For p parameters and m steps, the method reduces Hamiltonian launches from $(1 + 2p)m$ to

$$m + 2m\lceil p/q \rceil,$$

while retaining $O((q+1)n+m^2)$ working storage and the same $O(pmn)$ column-equivalent arithmetic for sparse operators. Full blocking ($q = p$) uses $3m$ launches independent of p . We prove, by a columnwise induction on the differentiated three-term recurrence, that every block width produces the exact derivative of the same finite Lanczos objective in exact arithmetic. Consequently any certified projection error bound for scalar replay transfers without inflation.

Four checksum-pinned PARSEC Hamiltonians from the SuiteSparse Matrix Collection provide authenticated sparse-matrix tests. Across four matrices and four depths, the largest Euclidean gradient difference between full blocking and parameterwise replay is 2.50×10^{-16} . At $p = 3$ and $m = 32$, nine alternating CPU repetitions give median speedups of 1.10, 1.21, 1.25, and 1.06 on matrices of dimension 769, 5,041, 8,219, and 19,896. A parameter sweep on SiH4 shows the explicit tradeoff: at $p = 8$, launches fall from 408 to 72 and median time falls from 0.224 to 0.103 seconds, while the peak ambient-vector ledger rises from 10.46 to 38 vector equivalents. At smallest tested parameter count has little work to amortize, so launch reduction is not stated as a universal wall-time guarantee. The contribution is an exact launch–memory continuum for matrix-free gradients, supported by executable operator ledgers, accuracy tests, and real sparse matrices.

Key words. Lanczos method, matrix-function derivative, block sparse multiplication, matrix-free calibration, operator-launch complexity, SuiteSparse

1. Introduction. Krylov projection reduces a large matrix-function action to a small dense problem. For a Hermitian matrix $A(\theta)$, a probe u , and an evolution time t , consider

$$q(\theta) = |u^* \exp[-itA(\theta)]u|^2. \quad (1.1)$$

The Lanczos approximation replaces A by an $m \times m$ tridiagonal $T_m(\theta)$. Matrix-function and Fréchet-derivative analyses for such projections are well established [1, 2, 4, 6]. A practical gradient still depends on the organization of the differentiated recurrence.

Scalar tangent replay retains a constant number of ambient vectors while it reconstructs the derivative $\dot{T}_{m,a}$ for parameter a . Repeating that replay for p parameters avoids an $n \times m$ basis but repeats identical primal operator applications. On accelerators, distributed systems, and sparse CPU kernels, launch count and memory traffic can matter independently of the formal floating-point count. The natural alternative is to propagate several tangents as columns of one multivector.

This paper develops that alternative as a tunable algorithm rather than as a fixed full batch. Block width $q = 1$ recovers parameterwise replay. Width $q = p$ minimizes Hamiltonian launches. Intermediate widths occupy the launch–memory frontier. The exactness proof does not appeal to favorable spectra or low tangent rank; it follows from the separability of the differentiated scalar recurrence. The method has no additional truncation parameter and does not form the ambient Krylov basis or the Hamiltonian.

The evidence is organized around four claims. Figure 1 compares block and scalar gradients across matrix and depth. Figure 2 reports alternating wall times on every authenticated matrix. Figure 3 shows the exact launch and storage ledgers as parameter count grows. Figure 4 tests whether those ledgers translate to runtime. Tables identify the matrices and give the numerical results needed to audit each claim.

2. Finite Lanczos objective. Let $v_1 = u/\|u\|_2$, $v_0 = 0$, and $\beta_0 = 0$. The Hermitian Lanczos recurrence is

$$w_j = Av_j - \beta_{j-1}v_{j-1}, \quad (2.1)$$

$$\alpha_j = v_j^* w_j, \quad (2.2)$$

$$r_j = w_j - \alpha_j v_j, \quad (2.3)$$

$$\beta_j = \|r_j\|_2, \quad v_{j+1} = r_j/\beta_j. \quad (2.4)$$

The resulting tridiagonal contains α_j on its diagonal and β_j on its first off-diagonals. The finite objective is

$$q_m(\theta) = \|u\|_2^4 |e_1^* \exp[-itT_m(\theta)]e_1|^2. \quad (2.5)$$

If $G_m = \nabla_{T_m} q_m$ denotes the symmetric projected sensitivity, then

$$\partial_a q_m = \text{tr}(G_m^T \dot{T}_{m,a}), \quad \dot{T}_{m,a} = \frac{\partial T_m}{\partial \theta_a}. \quad (2.6)$$

The small matrix G_m is obtained by a divided-difference evaluation of the matrix exponential. Only the construction of all $\dot{T}_{m,a}$ concerns the large dimension.

2.1. Differentiated recurrence. Write $D_a = \partial A/\partial \theta_a$ and place a dot on a quantity differentiated in direction a . Differentiating (2.4) gives

$$\dot{w}_{j,a} = D_a v_j + A \dot{v}_{j,a} - \dot{\beta}_{j-1,a} v_{j-1} - \beta_{j-1} \dot{v}_{j-1,a}, \quad (2.7)$$

$$\dot{\alpha}_{j,a} = 2 \text{Re}(\dot{v}_{j,a}^* A v_j) + v_j^* D_a v_j, \quad (2.8)$$

$$\dot{r}_{j,a} = \dot{w}_{j,a} - \dot{\alpha}_{j,a} v_j - \alpha_j \dot{v}_{j,a}, \quad (2.9)$$

$$\dot{\beta}_{j,a} = \text{Re}(v_{j+1}^* \dot{r}_{j,a}), \quad (2.10)$$

$$\dot{v}_{j+1,a} = \frac{\dot{r}_{j,a} - \dot{\beta}_{j,a} v_{j+1}}{\beta_j}. \quad (2.11)$$

The diagonal and off-diagonal entries of $\dot{T}_{m,a}$ are $\dot{\alpha}_{j,a}$ and $\dot{\beta}_{j,a}$.

3. Parameter-blocked replay. For an index set I of at most q parameters, collect tangent vectors into

$$\dot{V}_j^{(I)} = [\dot{v}_{j,a_1} \quad \cdots \quad \dot{v}_{j,a_{|I|}}] \in \mathbb{C}^{n \times |I|}. \quad (3.1)$$

The primal vector v_j is replayed once. The call $A \dot{V}_j^{(I)}$ is one block application, while the columns $D_a v_j$ are assembled into a derivative block. All inner products in (2.11) become columnwise reductions. After completing the block, the algorithm contracts its tridiagonal tangents with G_m and proceeds to the next parameter block.

Theorem 1 (Exact block equivalence). *Assume exact arithmetic, no Lanczos breakdown through step m , and an operator routine satisfying*

$$A[x_1, \dots, x_s] = [Ax_1, \dots, Ax_s]. \quad (3.2)$$

For every block width $q \geq 1$, parameter-blocked replay returns exactly the same $\dot{T}_{m,a}$ and $\partial_a q_m$ as parameterwise tangent replay for all $a = 1, \dots, p$.

Proof. Partition the parameters into disjoint ordered blocks. Fix a block I . At $j = 1$, both algorithms use

$$\dot{v}_{0,a} = \dot{v}_{1,a} = 0, \quad \dot{\beta}_{0,a} = 0, \quad (3.3)$$

for each $a \in I$. Thus corresponding scalar and block columns agree. Assume agreement through step j . By (3.2), column a of the block application $A \dot{V}_j^{(I)}$ is precisely $A \dot{v}_{j,a}$. The derivative block has column $D_a v_j$. Every remaining operation in (2.11) is an addition, scalar multiplication, or columnwise inner product. Hence the block algorithm produces the same $\dot{\alpha}_{j,a}$, $\dot{r}_{j,a}$, $\dot{\beta}_{j,a}$, and $\dot{v}_{j+1,a}$ as scalar replay. Induction proves equality through m . The entries of $\dot{T}_{m,a}$ therefore agree, and (2.6) gives identical gradient components. $\square$

Figure 1 is a floating-point audit of this theorem. It is not used as its proof.

Theorem 2 (Launch-storage frontier). *Let $s = \lceil p/q \rceil$. Counting an application to a vector or a multivector as one operator launch, parameter-blocked replay uses*

$$L_H(m, p, q) = m + 2ms \quad (3.4)$$

Hamiltonian launches and pm derivative-vector applications. Its large working storage is $O((q+1)n)$, and its small storage is $O(pm + m^2)$. The column-equivalent Hamiltonian work is $(p+2)m$ vector applications.

Proof. The initial Lanczos pass uses m vector launches. Each parameter block replays Av_j and $A\dot{V}_j^{(I)}$ at every step, contributing $2m$ launches. There are s blocks, proving (3.4). Each parameter contributes one $D_a v_j$ application per step, hence pm . At most a constant number of primal vectors and a constant number of $n \times q$ tangent blocks coexist, giving $O((q+1)n)$. Storing tangent tridiagonal coefficients requires $O(pm)$ scalars and G_m requires $O(m^2)$. A full block application contains at most q vector columns; summing columns over all blocks yields pm tangent columns, in addition to $2m$ primal columns. $\square$

Corollary 3 (Endpoints). *Parameterwise replay ($q = 1$) uses $(1+2p)m$ launches and $O(n)$ large storage. Full blocking ($q = p$) uses $3m$ launches and $O(pn)$ large storage.*

Proof. Substitute $q = 1$ and $q = p$ into Theorem 2. $\square$

Corollary 4 (Inherited projection certificate). *Suppose a finite Lanczos gradient g_m satisfies a certified unprojected bound*

$$\|g - g_m\|_2 \leq B_m. \quad (3.5)$$

Then every parameter-blocked result $g_m^{(q)}$ satisfies

$$\left\| g - g_m^{(q)} \right\|_2 \leq B_m. \quad (3.6)$$

Proof. Theorem 1 gives $g_m^{(q)} = g_m$ in exact arithmetic, so the two left-hand sides are equal. No block-width term enters the projection certificate. $\square$

4. Authenticated sparse-matrix study. The test matrices are real-symmetric electronic-structure Hamiltonians from the PARSEC group of the SuiteSparse Matrix Collection [3, 5]. Archives are downloaded from registered URLs and accepted only after SHA-256 verification. Local diagonal projectors define transparent parameter directions; they are not device measurements. Table 1 lists the source matrices.

Table 1: Authenticated PARSEC matrices. Dimensions and nonzero counts refer to the decoded symmetric matrices.

Matrix	n	nnz	SHA-256 prefix
Si2	769	17,801	84e9c6f4
SiH4	5,041	171,903	803b5243
benzene	8,219	242,669	a1f78f39
Si5H12	19,896	738,598	895c0d8c

All accuracy tests use $p = 3$, $t = 8$, a fixed normalized probe, and depths $m \in \{8, 16, 32, 48\}$. The reference is the independently executed parameterwise replay. Timing uses $m = 32$, nine alternating scalar-block repetitions, and a full parameter block. The parameter sweep uses SiH4, $m = 24$, $p \in \{1, 2, 3, 4, 6, 8\}$, and five alternating repetitions. Medians and 0.1–0.9 timing quantiles are reported; no timing value enters the exactness contract.

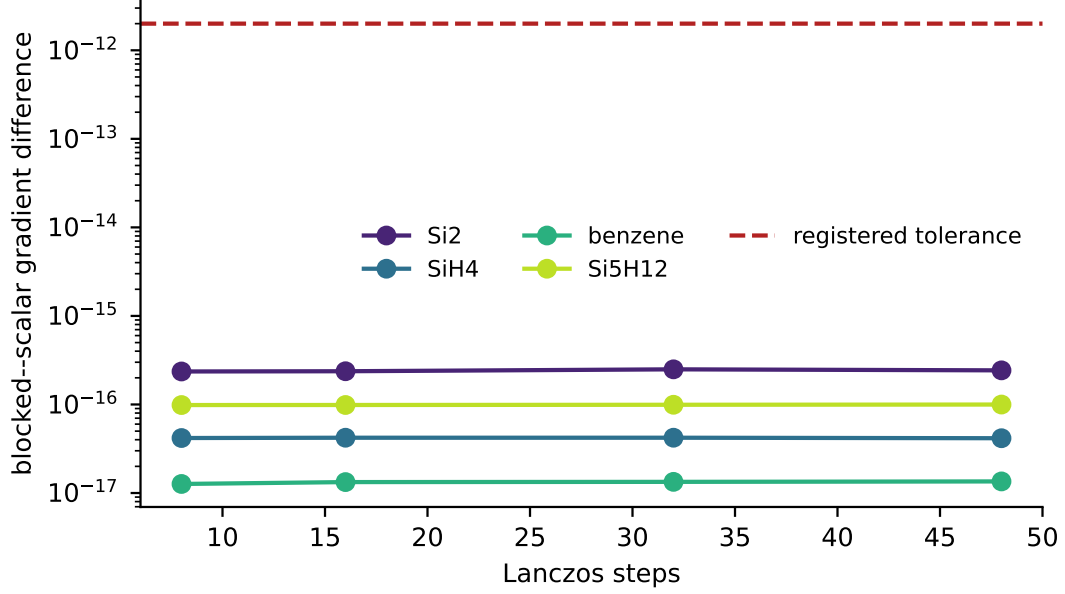


Fig. 1: Finite-objective equivalence across four authenticated matrices and four depths. The ordinate is the Euclidean difference between full-block and parameterwise gradients. Every value is at roundoff scale and below the registered 2×10^{-12} tolerance.

The maximum gradient difference in Figure 1 is 2.50×10^{-16} ; the maximum probability difference is zero at the reported precision. Every scalar run uses $7m$ Hamiltonian launches and every full-block run uses $3m$, exactly matching Corollary 3.

5. Runtime and parameter scaling. Table 2 gives the median paired timings. Full blocking is faster on all four matrices at $p = 3$, with speedups from 1.06 to 1.25. These are host-specific measurements rather than asymptotic guarantees.

Table 2: Alternating CPU timings at $p = 3$ and $m = 32$.

Matrix	Scalar (s)	Block (s)	Speedup
Si2	0.0122	0.0111	1.10
SiH4	0.0974	0.0803	1.21
benzene	0.1404	0.1127	1.25
Si5H12	0.2727	0.2575	1.06

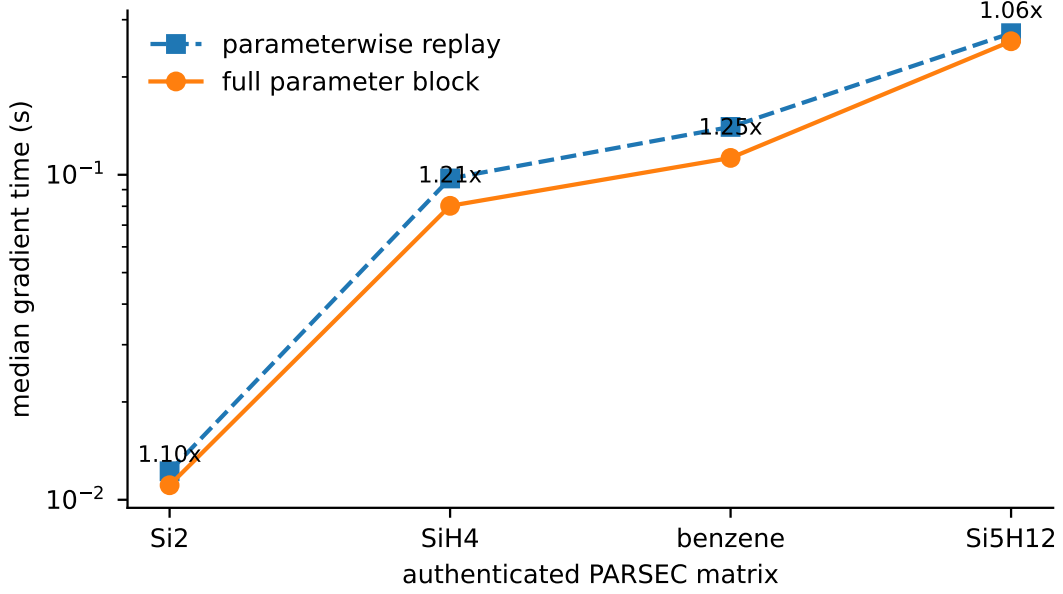


Fig. 2: Median gradient time from nine alternating repetitions. Labels give the scalar-to-block ratio. Accuracy is checked separately in Figure 1.

The launch reduction has a visible but smaller wall-time effect because a block sparse multiplication still moves and multiplies all p tangent columns. It benefits from a shared sparse traversal and fewer interpreter and kernel entries; it does not remove the column-equivalent arithmetic.

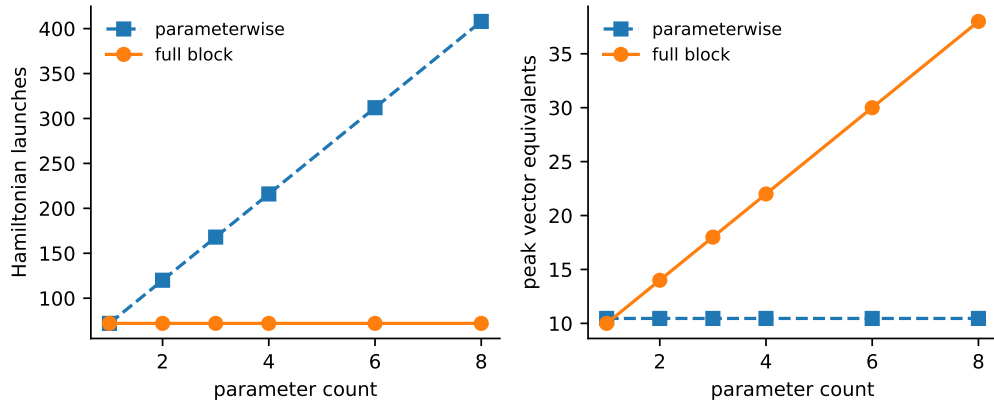


Fig. 3: Exact ledger on SiH4 at $m = 24$. Full blocking holds Hamiltonian launches at $3m = 72$ as p grows, compared with $(1 + 2p)m$ for parameterwise replay. The reduction is purchased with an ambient-vector ledger that grows linearly in p .

Figure 3 is the computational visualization of Theorem 2. At $p = 8$, scalar replay uses 408 launches and 10.46 vector equivalents in the released allocation model. Full blocking uses 72 launches and 38 vector equivalents. Intermediate q values permit a memory cap to be imposed without changing the finite gradient.

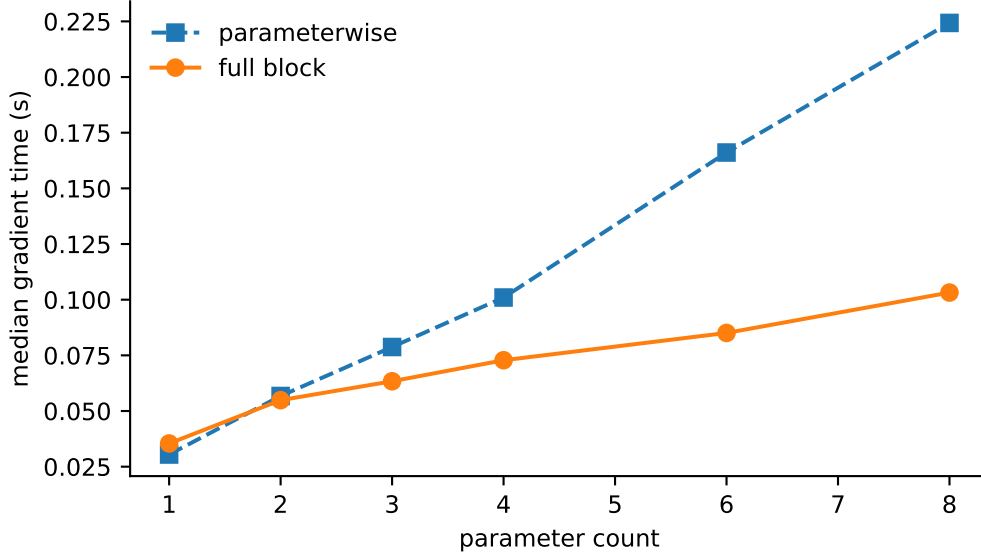


Fig. 4: Measured SiH4 parameter scaling. Full blocking gives a small timing difference at one and two parameters and an increasing separation thereafter. The curve is a host-specific consequence of the exact launch ledger.

At $p = 8$, median time decreases from 0.224 to 0.103 seconds. At $p = 2$, it changes from 0.0568 to 0.0549 seconds. Launch count is therefore a useful mechanistic predictor, not a sufficient performance theorem. The measured crossover depends on sparse-kernel implementation, matrix size, and memory hierarchy.

6. Limitations and reproducibility. The method assumes that the Hamiltonian action accepts multivectors and implements columnwise linearity. A backend that decomposes every block into separate vector calls can preserve exactness while losing the launch benefit. Large p can make full blocking memory prohibitive; the purpose of q is to expose that constraint. Floating-point equality is not asserted, although the observed discrepancies remain below 2.50×10^{-16} in the locked study. Loss of Lanczos orthogonality and matrix-function projection error are separate from parameter blocking.

The installable package under `code/` contains the new block replay, scalar comparator, authenticated downloader, tests, registered study, and figure generator. The command

```
rc-krylov-block-study --verify
```

reruns the accuracy and operator-ledger contract and regenerates every figure. The semantic result hash is

```
3dd1bab58150136019fd61ac6ceed799f152dabdc194d68ce61145dc85770213.
```

Timing fields are retained in the result but excluded from semantic equality. Twenty-three unit tests check the original recurrence, projected derivatives, certificates, block equivalence, block-width endpoints, and launch ledgers.

7. Conclusion. Parameter-blocked tangent replay reorganizes an exact finite Lanczos gradient without introducing an approximation. It spans the scalar low-memory endpoint and the full-block low-launch endpoint through one block-width parameter. The launch count falls from $(1 + 2p)m$ to $m + 2m\lceil p/q \rceil$, while storage grows with q . Authentic PARSEC tests confirm roundoff-level equivalence and show full-block speedups on all four matrices at three parameters. The parameter sweep also records the small benefit at low parameter counts. The resulting contribution is a proved and measured launch-memory frontier, not a blanket speed claim.

Data and code availability. Code, source URLs, checksums, numerical ledgers, timing records, tests, and figure scripts are included under `code/`. PARSEC archives remain at the SuiteSparse Matrix Collection and are not redistributed by this repository.

References.

- [1] Awad H. Al-Mohy and Nicholas J. Higham. Computing the Fréchet derivative of the matrix exponential, with an application to condition number estimation. *SIAM Journal on Matrix Analysis and Applications*, 30(4):1639–1657, 2009. doi: 10.1137/080716426.
- [2] Angelo A. Casulli and Igor Simunec. A low-memory Lanczos method with rational Krylov compression for matrix functions. *SIAM Journal on Scientific Computing*, 47(3):A1358–A1382, 2025. doi: 10.1137/24M1644699.
- [3] Timothy A. Davis and Yifan Hu. The university of Florida sparse matrix collection. *ACM Transactions on Mathematical Software*, 38(1):1:1–1:25, 2011. doi: 10.1145/2049662.2049663.
- [4] Nicholas J. Higham. *Functions of Matrices: Theory and Computation*. Society for Industrial and Applied Mathematics, Philadelphia, 2008. doi: 10.1137/1.9780898717778.
- [5] Scott Kolodziej, Mohsen Aznaveh, Matthew Bullock, Jarrett David, Timothy A. Davis, Matthew Henderson, Yifan Hu, and Read Sandstrom. The SuiteSparse matrix collection website interface. *Journal of Open Source Software*, 4(35):1244, 2019. doi: 10.21105/joss.01244.
- [6] Yousef Saad. Analysis of some Krylov subspace approximations to the matrix exponential operator. *SIAM Journal on Numerical Analysis*, 29(1):209–228, 1992. doi: 10.1137/0729014.